

VIGIE — Manuel technique

Document destiné à un public technique (administrateur système, futur développeur) — base de données, sécurité applicative et Row Level Security (RLS). Distinct du manuel utilisateur, rédigé pour un agent opérationnel du GPMM.

État au 23/09/2026.

1. Architecture générale

VIGIE est une application métier interne du Grand Port Maritime de Marseille (GPMM), construite sur la pile suivante :

- **Frontend** : React 18 + TypeScript, bundler Vite, routage React Router, état global via Context API. Une seule page par écran (`frontend/src/pages/`), composants réutilisables sans appel réseau direct (`frontend/src/components/`), tout appel HTTP centralisé dans `frontend/src/services/api.ts` .
- **Backend** : Node.js + Express + TypeScript, architecture en couches strictes : `routes` → `controllers` → `services` → `repositories` → Supabase . Un controller n'appelle jamais un repository directement ; un repository ne contient jamais de logique métier (uniquement des requêtes Supabase) ; la logique métier et la validation (schémas Zod) vivent dans la couche service.
- **Base de données et authentification** : Supabase (PostgreSQL managé + Supabase Auth). Toutes les tables métier de VIGIE vivent dans le schéma Postgres `finances` (jamais `public`), à l'exception de `public.profiles` , table de liaison identité partagée avec d'autres applications du même projet Supabase (voir §3.5).
- **Déploiement** : Railway (backend), configuration dans `backend/railway.json` . Docker Compose pour le développement local (frontend + backend en parallèle).
- **Tests** : Vitest partout — React Testing Library côté frontend, Supertest côté backend.

Le projet Supabase est **partagé entre plusieurs applications métier du GPMM**, isolées par schéma PostgreSQL : VIGIE dans `finances` , au moins une autre application (« escales ») dans son propre schéma. Toute intervention sur ce projet Supabase doit tenir compte de cette mutualisation — ne jamais écrire, modifier ni même interroger à titre exploratoire un objet d'un schéma appartenant à une autre application.

2. Base de données

2.1 Vue d'ensemble du schéma `finances`

Le schéma compte une vingtaine de tables, organisées en quatre grands ensembles :

- **Référentiel organisationnel** : `direction` , `service` , `cellule` , `acteur` — hiérarchie fixe Direction → Service → Cellule → Acteur, chaque niveau avec un indicateur `actif` (archivage par désactivation, jamais de suppression physique sauf cas résiduel documenté).
- **Référentiels métier** (import PGI ou gestion autonome) : `site` / `sous_site` , `secteur` / `sous_secteur` (gisements géographique/technique), `cug` (Compte Unitaire de Gestion), `operation_investissement` , `marche` , `marche_tiers` , `marche_piece` , `fournisseur` / `contact` , `commande_pgi` , `investissement_piece` , `libelle_referentiel` (référentiel générique de listes fermées, par domaine).
- **Rôles applicatifs** : `role_attribution` (nommée ainsi, pas `role` , pour éviter la collision avec la notion native de rôle Postgres/Supabase), `suppleance` , `suppleance_audit` .
- **Cœur métier — demande d'achat (DA/FAD)** : `demande_achat` , `devis_consulte` , `piece_jointe` , `statut` (référentiel des 25 codes de statut), `historique_statut` . Le circuit Service Fait (Phase 2, non implémenté à ce jour) ajoute `certificat_service_fait` , `statut_csf` , `historique_statut_csf` .

- **Paramétrage applicatif** : `parametre_application` , `parametre_definition` , `seuil_validation_ds` , `signature_acteur` .

Aucune table de jointure N:M dans l'ensemble du schéma — toutes les associations sont portées par des clés étrangères simples.

2.2 Le cycle DA/FAD (demande_achat)

La table centrale de l'application. Une demande d'achat (DA) naît en brouillon (`DA_EN_PREPARATION`), se transmet au circuit de validation où elle devient FAD (Fiche d'Achat) dès sa transmission au N+2 (CDS), traverse jusqu'à 4 paliers de validation (RC → CDS → CB → DS, le DS étant conditionnel au seuil de validation), puis débouche sur une commande. Le statut courant est dénormalisé sur `demande_achat.code_statut` , mais **synchronisé automatiquement par trigger** (`finances.trg_sync_statut_courant`) à chaque insertion dans `historique_statut` — la modification directe de cette colonne est bloquée en base (`REVOKE UPDATE (code_statut) ... FROM service_role`), le seul chemin possible pour faire évoluer le statut d'une DA/FAD est d'insérer une ligne dans `historique_statut` .

`historique_statut` est **immuable** : aucune ligne n'est jamais modifiée après création (migration `20260914160000_verrouiller_immutabilite_historique_statut.sql`), y compris son champ `commentaire_statut` — un texte de réponse à un motif antérieur se traduit toujours par une nouvelle ligne, jamais un ajout à la ligne existante.

Le référentiel `finances.statut` (25 codes) porte, pour chaque statut, les colonnes `pour_action` (rôle à qui une décision est explicitement demandée), `diffusion` (rôles informés) et `en_transit` (rôle qui détient actuellement l'objet — c'est ce champ, et lui seul, qui alimente la file « à traiter » de chaque rôle).

2.3 Périmètres de rôle

Chaque rôle applicatif (`role_attribution.type_role`) porte un périmètre différent, contraint par une clause `CHECK` en base :

Rôle	Périmètre	Colonne portée
RC	Cellule	<code>id_cellule</code>
CDS, CB, ADMIN_SERVICE	Service	<code>id_service</code>
DS	Direction	<code>id_direction</code>
ADMIN_APP	Transverse	(aucune)

Cette distinction a une conséquence directe sur l'implémentation : le périmètre du DS (une direction) peut couvrir plusieurs services, alors que `demande_achat` ne porte que `id_service` — toute requête scopée pour un DS nécessite une résolution en deux temps (direction → liste des services de cette direction → filtre sur cette liste), contrairement à CDS/CB qui filtrent directement sur un service unique.

Un même acteur peut cumuler plusieurs rôles (RC + CDS, CDS + CB, etc. — acceptée sans garde-fou en Phase 1). Le backend résout le rôle effectif d'un appel via `resolveAccessContext` (`backend/src/services/demandeAchat.service.ts`), qui accepte un paramètre `roleHint` explicite pour lever l'ambiguïté quand un écran de suivi dédié (RC/CDS/CB/DS) doit forcer la résolution sur un rôle précis plutôt que suivre l'ordre de priorité par défaut.

3. Sécurité applicative

3.1 Identité et authentification

L'authentification passe par **Supabase Auth**. `public.profiles` (schéma `public`, partagée avec les autres applications du même projet Supabase) fait le lien entre `auth.users.id` et l'identité métier via une colonne `matricule` — référence applicative vers `finances.acteur.matricule`, **pas** une contrainte de clé étrangère physique (un lien FK cross-schéma depuis une table partagée vers le schéma d'une seule application créerait un couplage indésirable). Tant que `matricule` est `NULL`, l'utilisateur est authentifié mais ne dispose d'aucune autorisation métier.

Le backend résout systématiquement le matricule à partir du token Supabase (middleware `requireAuth`, monté en tête de toute nouvelle route sans exception) — jamais fait confiance à une valeur envoyée par le client.

3.2 Double couche : application + base

VIGIE applique une défense en profondeur à deux niveaux :

1. **Couche applicative (Express)** — la très large majorité des règles métier : validation Zod par opération d'écriture, contrôle du périmètre (`assertManagesService`, `assertCanActFor`, `assertHasEffectiveRole`), machine à états des transitions de statut. C'est cette couche qui porte l'essentiel de la logique, le frontend et le backend n'appelant jamais directement Supabase pour une table `finances.*` — tout passe par `service_role`, qui contourne à la fois la RLS et les GRANT.
2. **Couche base (RLS + GRANT)** — filet de sécurité, pas la ligne de défense principale : si une policy trop permissive était ajoutée par erreur, ou une RLS désactivée par mégarde sur une table, ce filet évite qu'une brèche applicative devienne immédiatement exploitable via une requête PostgREST directe (`GET https://<projet>.supabase.co/rest/v1/<table>` avec le JWT de l'utilisateur).

3.3 Rôles Postgres/Supabase utilisés

- **`service_role`** — utilisé exclusivement par le backend Express. Contourne la RLS par construction et n'est jamais exposé au frontend (règle absolue : la clé `service_role` ne doit jamais être utilisée côté frontend).
- **`anon` / `authenticated`** — rôles PostgREST standard. N'ont aucun usage légitime sur les tables `finances.*` (le frontend ne fait que de l'auth via `supabase.auth.*`) : leurs GRANT ont été durcis (`anon` perd tout privilège partout ; `authenticated` perd les verbes jamais utilisés via PostgREST) en plus de la RLS.
- **`claude_readonly`** — rôle dédié, lecture seule, pour l'audit/l'introspection du schéma par l'assistant IA du projet (jamais d'écriture, jamais `service_role` ni `postgres`). Chaîne de connexion dans `backend/.env`, variable `DATABASE_URL_READONLY`.

3.4 Row Level Security (RLS)

Toutes les tables du schéma `finances` ont la RLS activée, sans exception. La grande majorité n'a **aucune policy** — ce qui, en Postgres, signifie un refus total par défaut à tout rôle sauf `service_role` (RLS activée + zéro policy = accès bloqué, pas ouvert). C'est le cas de la plupart des tables du cœur métier (`acteur`, `role_attribution`, `suppleance`, `devis_consulte`, `piece_jointe`, etc.) : leur accès direct via PostgREST n'a jamais été un besoin légitime, tout passe par le backend.

Quelques tables ont reçu des policies `SELECT` explicites, en défense en profondeur (aucun accès direct frontend n'en dépend aujourd'hui, mais elles couvrent un accès direct hypothétique) :

- `finances.demande_achat` / `finances.historique_statut` — policy scoping via la fonction `security` définir `finances.can_view_demande_achat(id_demande_achat)` : demandeur propriétaire, ou rôle dont le périmètre couvre la DA/FAD (RC de la cellule du demandeur, CDS/CB/ADMIN_SERVICE du service, DS de la direction, ADMIN_APP transverse).
- `finances.statut` — lecture ouverte à tout utilisateur authentifié (référentiel transverse, aucune notion de périmètre).
- `finances.suppleance` / `finances.suppleance_audit` — lecture scoping via `finances.can_view_suppleance(id_role)`.
- `finances.operation_investissement`, `finances.investissement_piece` — lecture scoping par service.
- Les référentiels administrables (`site` / `secteur` et leurs sous-niveaux, `cug`, `libelle_referentiel`, `parametre_application`) portent aussi des politiques `INSERT` / `UPDATE` pour `ADMIN_APP` / `ADMIN_SERVICE`, en plus de la lecture.

Piège à connaître : toute vérification d'autorisation dans une policy RLS doit passer par une fonction `security` définir (jamais une sous-requête directe sur `profiles` / `role_attribution` / `suppleance` dans le corps d'une policy), pour éviter la récursion RLS. La fonction clé est `finances.current_user_has_role(p_type_role text, p_perimeter_id integer)` — **attention** : son paramètre `p_perimeter_id` est typé `integer`, alors que les colonnes de périmètre (`id_service`, `id_cellule`, `id_direction`) sont `bigint` en base ; Postgres ne caste jamais implicitement `bigint` vers `integer` (rétrécissement) — tout appel doit caster explicitement (`da.id_service::integer`), sous peine d'échec silencieux de la fonction (`function ... does not exist`).

3.5 Point de vigilance — `public.profiles`

`public.profiles` est la seule table touchée par VIGIE qui vit hors du schéma `finances`. Elle est **partagée avec d'autres applications GPM** (ex. « escales ») : ne jamais la modifier unilatéralement (GRANT, RLS, politiques) sans coordination avec qui gère l'autre application. Elle porte sa propre RLS et une policy `profiles_select_self` (lecture de sa propre ligne uniquement).

3.6 Fiabilité des migrations — point structurel non résolu

Aucune migration de ce dépôt n'a jamais été appliquée via `supabase db push` (le schéma de suivi `supabase_migrations.schema_migrations` n'existe pas sur ce projet) : tout changement de schéma a été appliqué à la main dans l'éditeur SQL du dashboard Supabase, sans trace fiable de ce qui a réellement été exécuté. Deux régressions concrètes en ont découlé le 23/09/2026 (RLS de `public.profiles` désactivée sans que personne ne l'ait décidé ; politiques de `demande_achat` / `historique_statut` / `statut` jamais réellement appliquées malgré leur présence dans le dépôt depuis le 15/09/2026).

Conséquence pratique : la présence d'un fichier dans `supabase/migrations/` ne garantit rien sur l'état réel de la base. Toute nouvelle migration doit être vérifiée après application manuelle (requête directe sur `pg_policies` / `pg_class.rerowsecurity`, ou consultation du Security Advisor Supabase). Tant que ce point n'est pas résolu (migration CLI fonctionnelle depuis un réseau qui n'en bloque pas le port, ou reconstitution de l'historique via `supabase migration repair`), c'est le principal risque de dérive silencieuse entre le dépôt et la base réellement servie aux utilisateurs.

3.7 Validation, injections, XSS

- Toute donnée entrante côté Express est validée par un schéma Zod explicite avant traitement ; rejet par défaut (400) sur toute donnée non conforme, jamais de correction silencieuse.
- Aucune concaténation de chaînes pour construire une requête SQL — client Supabase (`supabase-js`) ou requêtes paramétrées uniquement.

- `dangerouslySetInnerHTML` n'est utilisé qu'à un seul endroit du frontend (`pages/Manuel.tsx` , pour le contenu HTML statique du manuel en ligne, rédigé à la main par l'équipe — jamais une saisie utilisateur) ; partout ailleurs, toute donnée provenant de l'API est traitée comme non fiable, même issue de la propre base de l'application.
 - CORS strict côté Express (liste blanche explicite via `FRONTEND_URL` , jamais `origin: '*'`) — un mauvais réglage local classique : accéder à l'application via `127.0.0.1` alors que `FRONTEND_URL` n'autorise que `localhost` bloque silencieusement toutes les requêtes API (le navigateur les traite comme une autre origine).
-

4. Pour aller plus loin

Ce document est une synthèse. Le détail complet, avec les dates de décision et le raisonnement associé à chaque règle, se trouve dans :

- `ForClaude/SECURITY.md` — consignes de sécurité exhaustives, section par section.
- `ForClaude/CDC/ml-d-phases-1-2.md` — modèle logique de données complet (schéma relationnel, contraintes d'intégrité).
- `docs/ARCHITECTURE.md` — architecture applicative détaillée.